Agent 1 - Monitoring & détection d'anomalies

Rapport de travail

Projet : Intelligent Multi-Agent System for Autonomous DevOps / MLOps Pipelines

Farah Marzougui - Member 1 - ESPRIT

Branche : Agent-1---Monitoring-&-Anomaly-Detection

Octobre 2026

Résumé

L'Agent 1 surveille en continu les services d'une plateforme (métriques Prometheus, logs Loki, conteneurs cAdvisor). Il détecte les anomalies, les regroupe en incidents, localise le service d'origine, décide s'il faut alerter et envoie à l'Orchestrateur un événement conforme au contrat v1 de l'équipe. Après une réparation, il vérifie lui-même la guérison. Il a été validé par des pannes réelles sur un banc de test, sur une chaîne de services et sur l'API de détection de fraude choisie par l'équipe, où il a mis en évidence deux bugs.

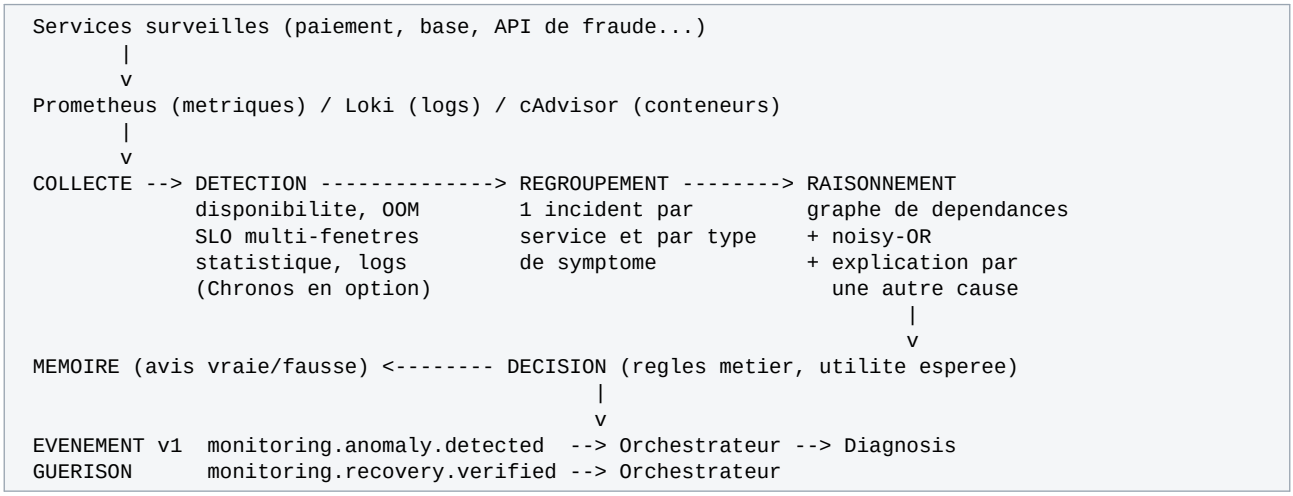
1. Rôle de l'agent

Dans le système à six agents, l'Agent 1 est le premier maillon : il observe et signale. Il répond à cinq questions : **quoi** (type de symptôme), **où** (service), **quand**, **quelle gravité** et **quelle confiance**. Il ne donne jamais la cause (rôle de l'agent Diagnosis) et n'agit jamais sur les services (rôle de l'agent Deployment-Recovery). Il est en lecture seule.

L'Agent 1 fait	L'Agent 1 ne fait pas
Observer les métriques, les logs et les conteneurs	Modifier le code ou la configuration des services
Détecter, regrouper, localiser l'origine probable	Expliquer la cause racine
Alerter l'Orchestrateur (événement v1)	Réparer, redémarrer, revenir en arrière
Vérifier indépendamment une réparation	Juger la qualité des prédictions ML (Agent 6)

2. Architecture

Un cycle complet s'exécute toutes les 30 secondes (boucle autonome, `python -m src.agents.monitoring`). Chaque couche est testée séparément ; une source ou un détecteur en panne n'arrête jamais le cycle.



Couche	Fichier(s) et principe
Collecte	`collectors/` : requêtes PromQL/LogQL avec délai, nouvel essai et disjoncteur. Profil par service : mode *app* (noms de métriques configurables) ou *conteneur* (cAdvisor, sans instrumentation).
Disponibilité	`detectors/availability.py` : service sans réponse depuis 45 s : incident *readiness*, gravité HIGH. Prometheus indisponible n'est pas un service mort.
OOM	`detectors/oom.py` : conteneur tué par le noyau faute de mémoire (compteur cAdvisor) : *memory_oom*, CRITICAL (P0).
SLO	`detectors/slo.py` : objectif 99,5 %, latence < 500 ms. Trois règles de burn rate : very-fast 5 min/1 min (36), fast 1 h/5 min (14,4), slow 6 h/30 min (6).
Statistique	`detectors/statistical.py` : pente de la mémoire (régression, R^2), sauts de latence et de CPU (médiane, MAD, z robuste) par rapport aux 30 minutes précédentes.
Logs	`detectors/logs.py` : modèles Drain3, nouveau message ou explosion (×5), étiquettes fermées, secrets masqués, Gemini optionnel.
Prévision (option)	`detectors/forecast.py` : Chronos-Bolt (Amazon, pré-entraîné, CPU). Désactivé par défaut (voir 7).
Regroupement	`grouping.py` : un incident par service et type de symptôme. Retour après 5 min de silence = nouvel incident.
Raisonnement	`reasoning.py` + `configs/dependencies.yaml` : graphe de dépendances, noisy-OR, explication par une autre cause (localisation, pas cause).

Couche	Fichier(s) et principe
Décision	`decision.py` : utilité espérée (alerter / attendre / vérifier), règles métier (HIGH/CRITICAL toujours alertés), alerte sur l'origine probable, anti-répétition.
Mémoire	`memory.py`, `feedback.py` : historique SQLite des alertes, avis vraie/fausse de l'opérateur, précision Beta par service et symptôme.
Événements	`events.py` : EventEnvelope v1 + AnomalyDetected, priorité selon la gravité, clé d'idempotence ; fichier JSONL en attendant Kafka.
Guérison	`recovery.py` : après *deployment.action.completed*, mesures répétées : RECOVERED, NOT_RECOVERED ou INCONCLUSIVE.
Agent officiel	`service.py`, `inbox.py` : tâches *detect_anomaly* et *verify_recovery* de l'Orchestrateur.

3. Méthode de validation

Un agent de surveillance ne peut pas être évalué en attendant des pannes réelles : elles sont rares et leur cause est inconnue. La méthode retenue est l'ingénierie du chaos sur un environnement de test : des pannes contrôlées sont provoquées, la vérité est donc connue (type, service, heure), et l'on mesure si l'agent la retrouve et en combien de temps. L'agent n'a aucun accès aux scripts d'injection : il ne voit que les métriques et les logs.

Banc	Contenu	Ce qu'il valide
Service de test	faux service de paiement avec pannes sur commande (/chaos/memory, cpu, errors, latency)	chaque détecteur, les délais
Chaîne de services	payment-service -> payments-db (2 services), puis checkout-api -> payment-service -> payments-db (3 services)	le raisonnement (localisation de l'origine)
Projet réel de l'équipe	API FastAPI de détection de fraude (Fraud_Detection_MLOps), lancée depuis son propre dépôt, non modifiée	le comportement sur une vraie application
Tests automatiques	plus de 270 tests unitaires et de contrat (pytest, ruff, mypy strict, schémas JSON)	chaque règle, sans Docker

Principe

Les pannes sont provoquées uniquement sur des copies de test. En production, l'agent ne crée rien : il surveille et attend les vrais problèmes. La preuve qu'il n'est pas « programmé pour une panne » : avec les mêmes règles générales, il a détecté des pannes de types différents et deux bugs réels inconnus de l'API de fraude.

4. Résultats sur le service de test

Scénario complet du 11/10 (une panne par signal, agent relancé à chaque phase, cycle de 30 s) :

Panne injectée	Détecteur	Délai
Fuite mémoire 30 Mo/min	statistique (pente) / Chronos	2 min 50 s / 3 min 20 s
Latence +800 ms	SLO latence / statistique / Chronos	2 min 23 s / 2 min 53 s / 3 min 23 s
50 % d'erreurs HTTP 500	SLO disponibilité	5 min 21 s
CPU saturé	statistique / Chronos	2 min 52 s / 3 min 22 s

- Aucune alerte sur un signal sain pendant une panne d'un autre type.
- Vérification de guérison : **NOT_RECOVERED** quand l'action se dit réussie alors que la fuite continue, **RECOVERED** après la vraie réparation.
- Ces délais ont été mesurés avant l'ajout de la règle very-fast et du cycle de 30 s par défaut (section 7), qui les réduisent pour les pannes violentes.

5. Résultats sur l'API de détection de fraude

L'API est construite depuis son dépôt GitHub (profil Docker *fraud*) et reliée à l'agent par trois lignes de configuration (compose, Prometheus, monitoring). Scénario rejoué le 16/10, de 15:27 à 15:42 :

Action	Réaction de l'agent	Délai
Trafic normal	service UP, aucune alerte	-
Arrêt de l'API	readiness HIGH, alerte **P1**	62 s
Redémarrage + message de réparation	UP, puis vérification **RECOVERED**	27 s

Action	Réaction de l'agent	Délai
50 % des transactions déclenchent un bug	error_rate MEDIUM, alerte **P2**, puis « gravite en hausse » HIGH, alerte **P1**	53 s / 83 s

Deux bugs réels de l'API trouvés grâce à l'agent (HTTP 500)

1) *paymentMethodAgeDays* est converti en entier par l'API alors que la signature du modèle MLflow attend un réel : toute carte de plus d'un jour provoque une erreur 500. 2) Une *Category* absente (autorisée par le schéma) fait échouer la concaténation *Category* + "_" + *paymentMethod* dans *Src/model.py*. Le premier essai a été signalé par l'agent alors que l'auteur du test croyait envoyer du trafic sain.

6. Raisonnement sur une chaîne de services

Quand une base de données ralentit, le service qui l'appelle ralentit aussi : l'agent voit deux malades et doit désigner l'origine. Le raisonnement combine les indices par noisy-OR (*P = 1 - prod(1 - w)*) : symptôme propre, symptôme d'un service appelant, message de log pointant une dépendance.

Campagne en cours : résultats ajoutés à la fin des essais.

7. Améliorations issues des tests réels

Chaque défaut observé en vrai a été corrigé et protégé par un test qui échoue avec l'ancien code :

Défaut observé	Correction
Les erreurs récentes étaient cachées par la limite de 5 000 lignes de Loki	requête filtrée sur les lignes d'erreur, fenêtre par fenêtre
Fausse alerte CPU sur un service au repos (0 -> 0,02 cœur)	écart minimal significatif par signal
Une vraie fuite mémoire masquée par une alerte CPU déjà envoyée	nouvelle alerte sur nouveau symptôme
Service arrêté : seulement « données insuffisantes »	détecteur de disponibilité (readiness HIGH, P1)
MEDIUM devenu HIGH sans nouvelle alerte (resté en P2)	nouvelle alerte « gravite en hausse », suivie par type de symptôme
L'alerte citait une panne déjà réparée	événement limité aux incidents du cycle
L'alerte visait la victime (paiement) au lieu de l'origine (base)	alerte sur l'origine probable donnée par le raisonnement
Une nouvelle panne fusionnée avec l'ancienne (« deja alerte »)	retour après 5 min de silence = nouvel incident et nouvelle alerte
Alerte sur la victime 30 s avant que l'origine soit mesurée	attente d'un cycle si une dépendance soupçonnée n'est pas encore mesurée
Chaîne à 3 niveaux : le service du milieu aussi accusé que la base	explication par une autre cause (indice propre divisé par deux)
Détection trop lente des pannes violentes	cycle de 30 s, règle SLO very-fast (5 min/1 min), service mort en 45 s

Autres ajouts : incidents ouverts conservés après un redémarrage (SQLite), détection *memory_oom* (P0), outil de calibration des seuils sur un historique normal (`experiments/checks/calibrate.py`), Chronos rendu optionnel (il confirmait les pannes environ 30 s après la statistique, s'habituait aux pannes longues et coûtait environ 350 Mo de RAM).

8. Limites et suite

Limite	Situation
Kafka, Context Service	parties communes de l'équipe, non encore implémentées ; l'agent passe par les ports prévus (EventPublisher) : le branchement ne changera pas son code
Calibration des seuils	seuils issus des recommandations (Google SRE) et des tests ; l'outil de calibration demande plusieurs jours d'historique normal
Périmètre	Docker uniquement (pas de Kubernetes ni de traces)
Qualité des prédictions ML	non visible dans les métriques techniques : rôle de l'Agent 6
Délai de détection	volontairement non instantané (de 30 s à quelques minutes) pour éviter les fausses alertes ; réglable par service

Reproduire les résultats :

```
cd C:\Users\marzo\Downloads\agent1-repo
.venv\Scripts\python.exe -m pytest -q tests                        # tests automatiques
.venv\Scripts\python.exe experiments\checks\full_scenario.py       # service de test
.venv\Scripts\python.exe experiments\checks\fraud_scenario.py      # API de fraude
.venv\Scripts\python.exe experiments\checks\reasoning_scenario.py --chain 3
```